Cahier des charges techniques

Sommaire

1. Contexte du projet
    1.1. Présentation du projet
    1.2. Date de rendu du projet
2. Besoins fonctionnels
3. Ressources nécessaires à la réalisation du projet
    3.1. Ressources matérielles
    3.2. Ressources logicielles
4. Gestion du projet
5. Conception du projet
    5.1. Le front-end
        5.1.1. Wireframes
        5.1.2. Maquettes
        5.1.3. Arborescences
    5.2. Le back-end
        5.2.1. Diagramme de cas d'utilisation
        5.2.2. Diagramme d'activités
        5.2.3. Modèles Conceptuel de Données (MCD)
        5.2.4. Modèle Logique de Données (MLD)
        5.2.5. Modèle Physique de Données (MPD)
6. Technologies utilisées
    6.1. Langages de développement Web
    6.2. Base de données
7. Sécurité
    7.1. Login et protection des pages administrateurs
    7.2. Hachages des mots de passe avec Bcrypt
    7.3. Protection contre les attaques XSS (Cross-Site Scripting)
    7.4. Protection contre les injections SQL

1. Contexte du projet

1.1. Présentation du projet

Votre agence web a été sélectionnée par le comité d'organisation des jeux olympiques de Los Angeles 2028 pour développer une application web permettant aux organisateurs, aux médias et aux spectateurs de consulter des informations sur les sports, les calendriers des épreuves et les résultats des JO 2028.

Votre équipe et vous-même avez pour mission de proposer une solution qui répondra à la demande du client.

1.2. Date de rendu du projet

Le projet doit être rendu au plus tard le 18/12/2025.

2. Besoins fonctionnels

Le site web devra avoir une partie accessible au public et une partie privée permettant de gérer les données.

Les données seront stockées dans une base de données relationnelle pour faciliter la gestion et la mise à jour des informations. Ces données peuvent être gérées directement via le site web à travers un espace administrateur.

3. Ressources nécessaires à la réalisation du projet

3.1. Ressources matérielles

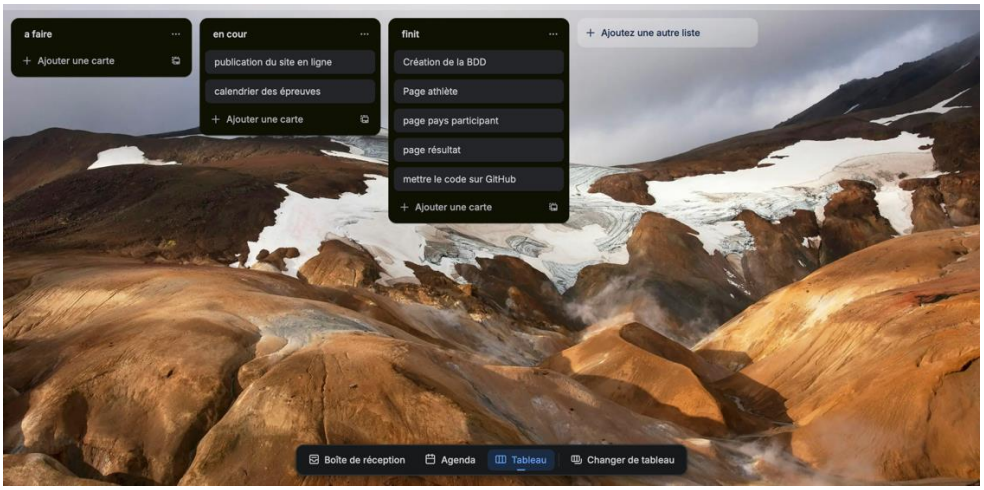
- PC
- Connexion internet
- Serveur local

3.2. Ressources logicielles

- VS Code
- PHP
- MySQL
- Mamp
- Mocodo
- Javascript
- HTML
- CSS
- trello

4. Gestion du projet

Pour réaliser le projet, nous utiliserons la méthode Agile Kanban. Nous utiliserons également l'outil de gestion de projet en ligne Trello.

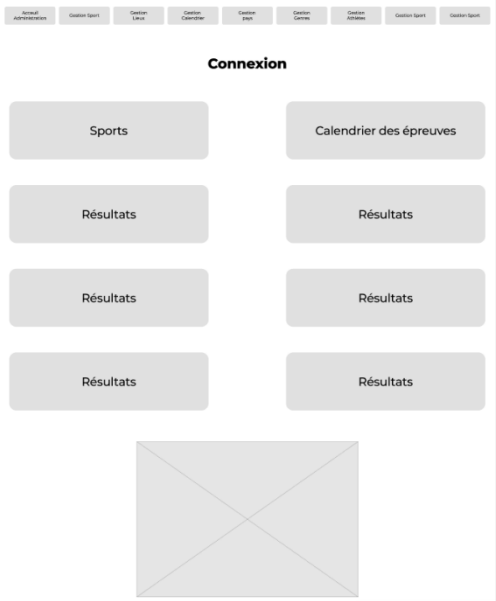


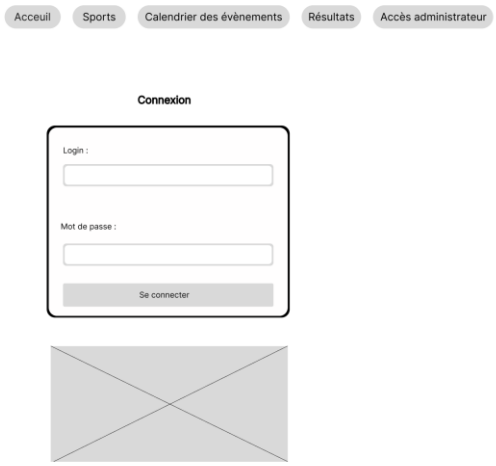
Nous travaillons également sur GitHub, plateforme de développement collaboratif.

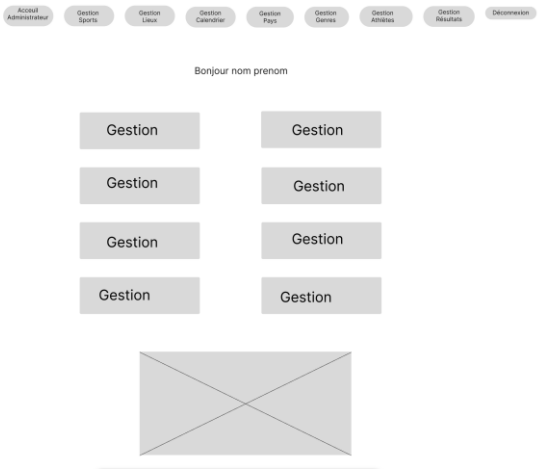
5. Conception du projet

5.1. Le front-end

5.1.1. Wireframes







5.1.2. Maquettes

Accueil Administration
Gestion Sports
Gestion Lieux
Gestion Calendrier
Gestion Pays
Gestion Genres
Gestion Athlètes
Gestion Résultats
Déconnexion

Accueil
Sports
Calendrier des épreuves
Résultats
Accès administrateur

Accueil
Sports
Calendrier des évènements
Résultats
Accès administrateur

Accueil Administration
Gestion Sports
Gestion Lieux
Gestion Calendrier
Gestion Pays
Gestion Genres
Gestion Athlètes
Gestion Résultats
Déconnexion

Connexion

Login :

Mot de passe :

Se connecter

Sports

Calendrier des épreuves

Résultats

Bonjour Super Admin

Gestion Administrateurs

Gestion Sports

Gestion Lieux

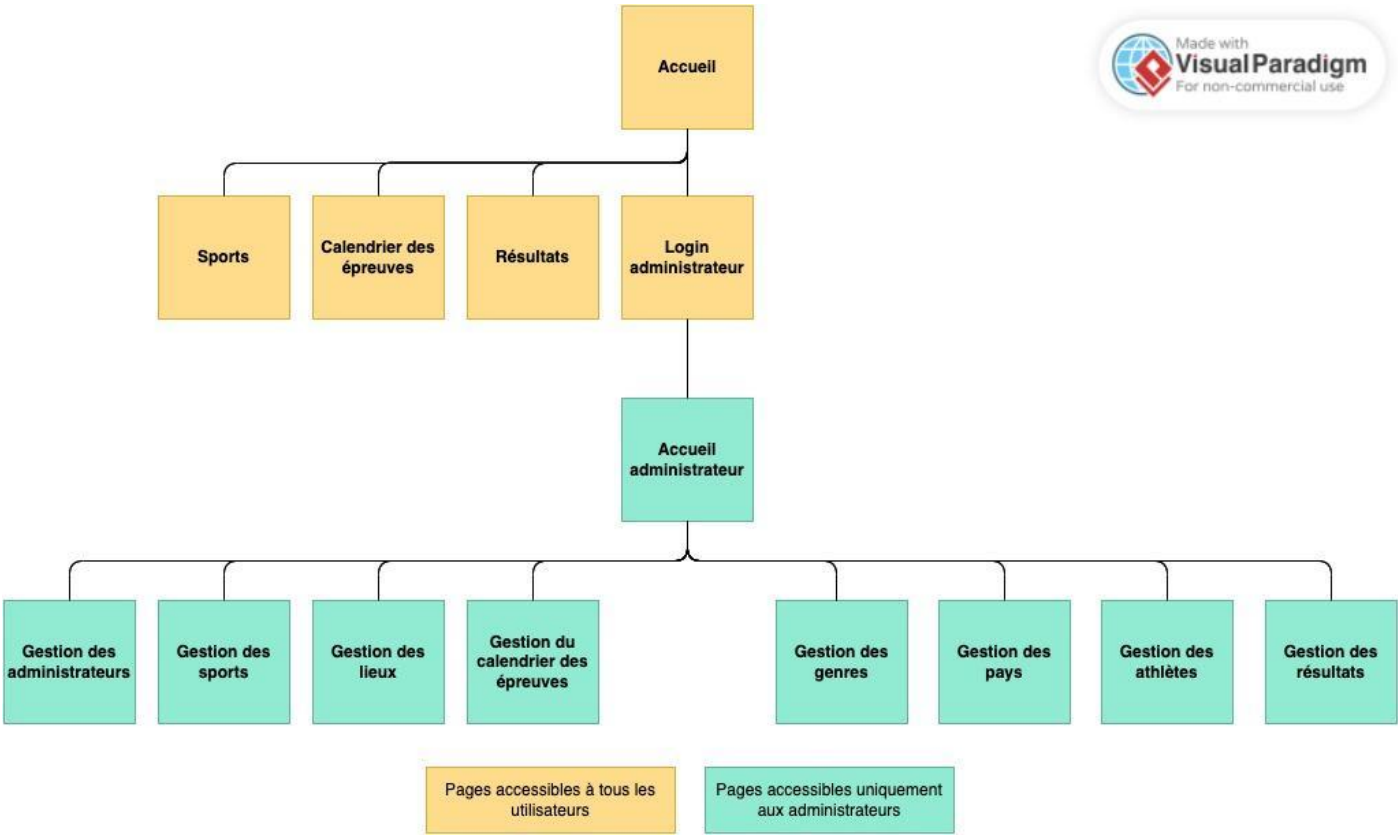
Gestion Calendrier

Liste des Sports

Ajouter un Sport

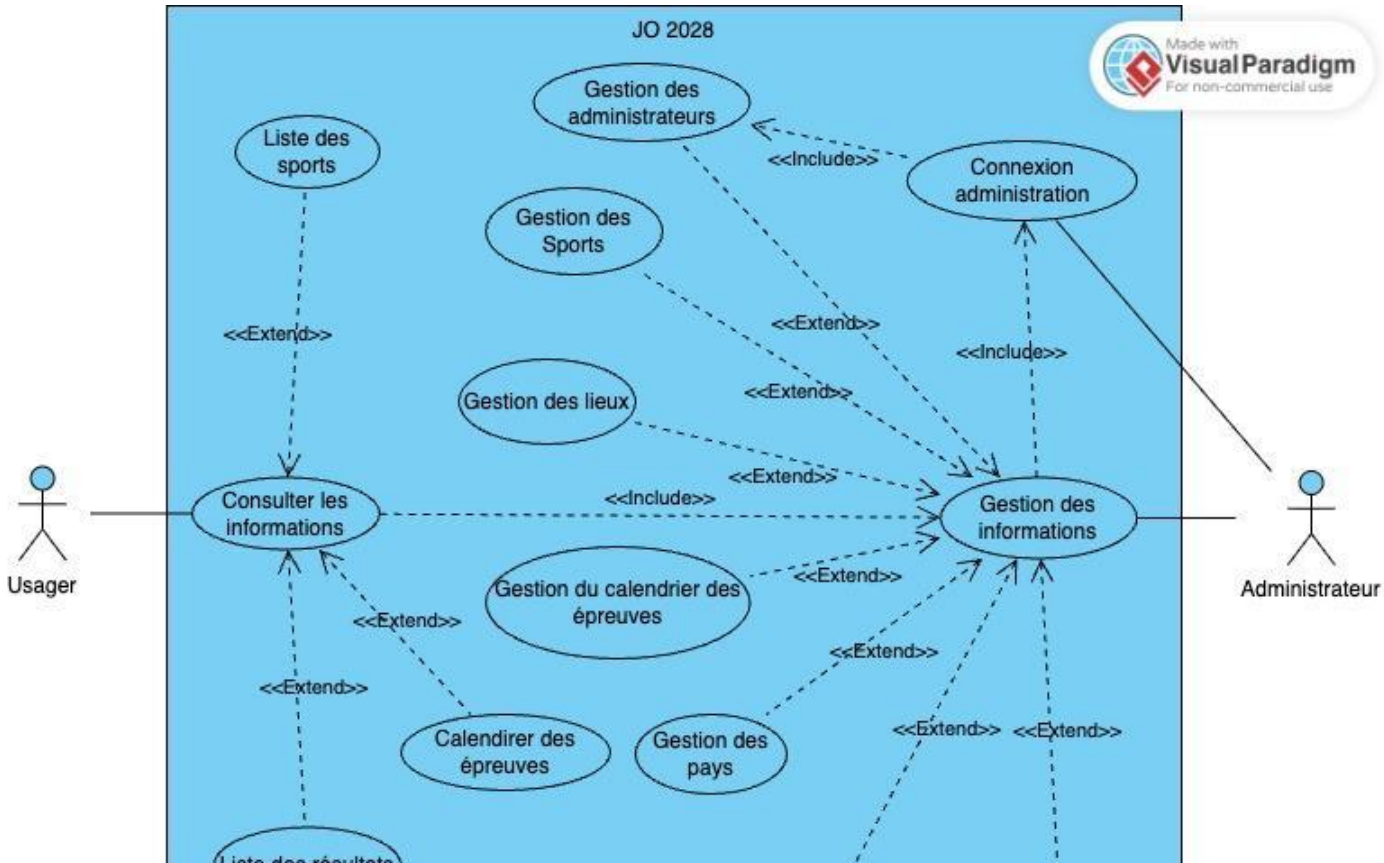
Sport	Modifier	Supprimer
Athlétisme	Modifier	Supprimer
Boxe	Modifier	Supprimer
Cyclisme	Modifier	Supprimer
Escalade	Modifier	Supprimer
Gymnastique	Modifier	Supprimer
Lancer	Modifier	Supprimer
Natation	Modifier	Supprimer

5.1.3. Arborescences

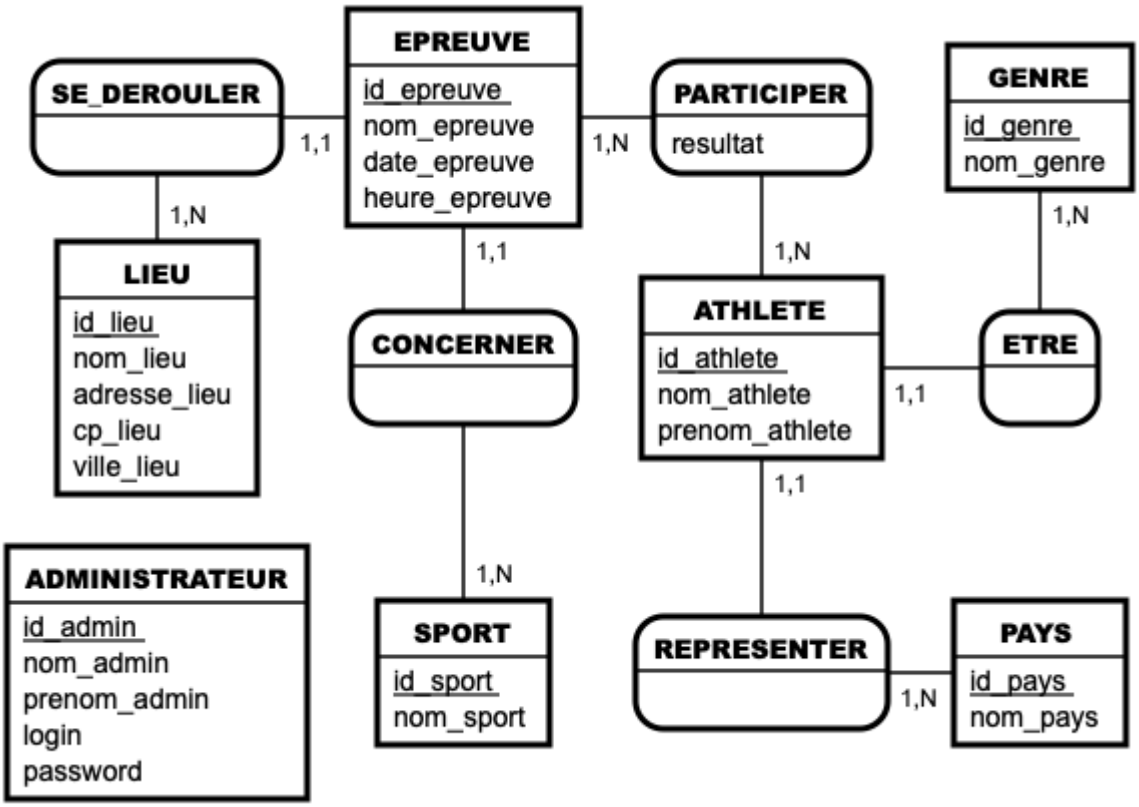


5.2. Le back-end

5.2.1. Diagramme de cas d'utilisation



5.2.3. Modèles Conceptuel de Données (MCD)



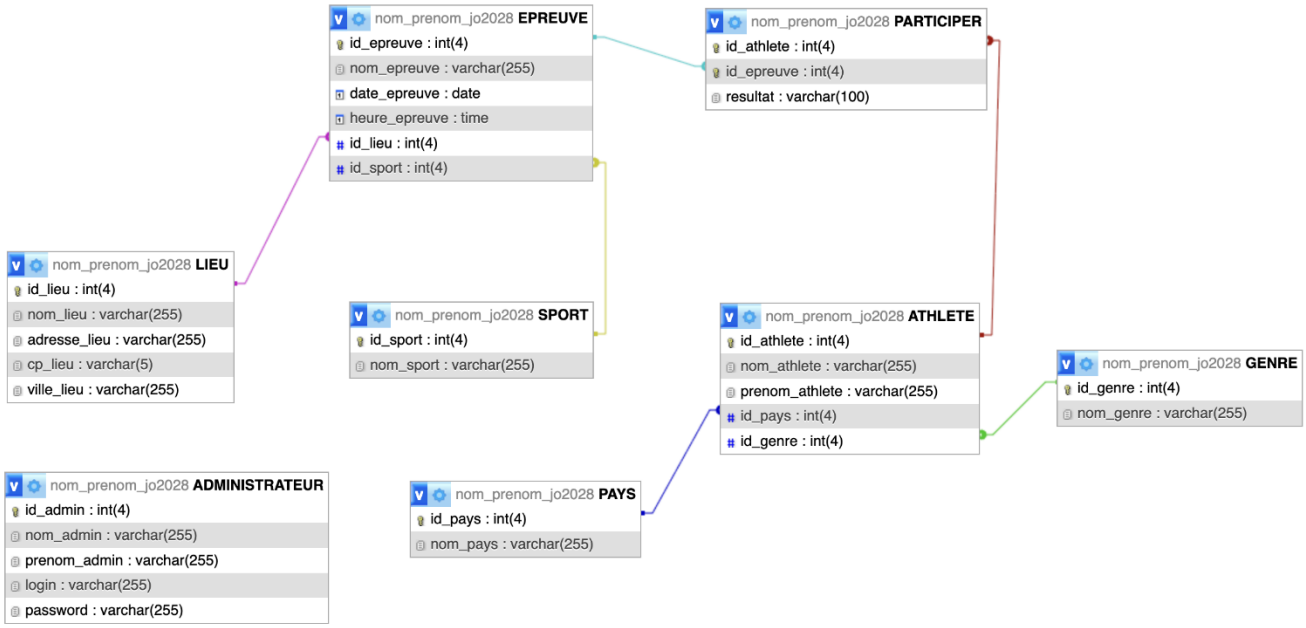
5.2.4. Modèle Logique de Données (MLD)

- ATHLETE (id_athlete, nom_athlete, prenom_athlete, id_pays, id_genre)
  Clé primaire : id_athlete

Clés étrangères : id_pays en référence à id_pays de PAYS id_genre en référence à i d_genre de GENRE

- EPREUVE (id_epreuve, nom_epreuve, date_epreuve, heure_epreuve, id_lieu, id_sport)
  Clé primaire : id_epreuve
  Clés étrangères : id_lieu en référence à id_lieu de LIEU id_sport en référence à id_sport de SPORT
- GENRE (id_genre, nom_genre)
  Clé primaire : id_genre
- LIEU (id_lieu, nom_lieu, adresse_lieu, cp_lieu, ville_lieu)
  Clé primaire : id_lieu
- PARTICIPER (id_athlete, id_epreuve, resultat)
  Clé primaire : id_athlete, id_epreuve
  Clés étrangères : id_athlete en référence à id_athlete de ATHLETE id_epreuve en référence à id_epreuve de EPREUVE
- PAYS (id_pays, nom_pays)
  Clé primaire : id_pays, id_epreuve
- SPORT (id_sport, nom_sport)
  Clé primaire : id_sport
- UTILISATEUR (id_utilisateur, nom_utilisateur, prenom_utilisateur, login, password)
  Clé primaire : id_utilisateur
- ADMINISTRATEUR (id_admin, nom_admin, prenom_admin, login, password)
  Clé primaire : id_admin

5.2.5. Modèle Physique de Données (MPD)



6. Technologies utilisées

`6.1. Langages de développement Web

- HTML (
- CSS
- JavaScript
- PHP

6.2. Base de données

- MOCODO
- MAMP
- MySQL

7. Sécurité

7.1. Login et protection des pages administrateurs

Pour protéger les pages admin, j'utilise les sessions PHP. Quand un utilisateur se connecte, une session

est créée. Chaque page admin vérifie si cette session existe, sinon l'utilisateur est redirigé.

```php
session_start();
require_once("../../database/database.php");

// Vérifiez si l'utilisateur est connecté
if (!isset($_SESSION['login'])) {
    header(header: 'Location: ../../index.php');
    exit();
```

7.2. Hachages des mots de passe avec Bcrypt

Le hachage sert à protéger les mots de passe en les transformant en une longue suite de caractères illisible.
Avec Bcrypt, on ne garde jamais le mot de passe réel dans la base de données.
Même si quelqu'un obtient la base, il ne pourra pas retrouver les mots de passe d'origine, car le hachage n'est pas réversible.

```php
    // HACHAGE DU MOT DE PASSE : ESSENTIEL POUR LA SÉCURITÉ
    $hashed_password = password_hash(password: $password, algo: PASSWORD_DEFAULT);
```

7.3. Protection contre les attaques XSS (Cross-Site Scripting)

Une attaque XSS injecte du JavaScript via des champs texte. Pour l'éviter, j'utilise htmlspecialchars(), qui transforme les caractères spéciaux en html avant d'afficher des données.

```php
<p class="info-login">
    Bonjour, <?php echo htmlspecialchars(string: $nom_utilisateur . " " . $prenom_utilisateur); ?>
</p>
```

7.4. Protection contre les injections SQL

Les injections SQL permettent d'insérer du code dans une requête pour la détourner.
Pour éviter ça, j'utilise des requêtes préparées avec PDO.

```php
    // Requête pour ajouter un sport
    $query = "INSERT INTO SPORT (nom_sport) VALUES (:param_nom_sport)";
    $statement = $connexion->prepare(query: $query);
    $statement->bindParam(param: ":param_nom_sport", var: &$nom_sport, type: PDO::PARAM_STR);
```